

CS 663: Digital Image Processing

Assignment 1: Problem 2(c) Report

Pushpendra Uikey

September 6, 2026

Contents

1	Thresholding	2
1.1	2(c) Local / Adaptive Thresholding (Niblack's Method)	2

1 Thresholding

1.1 2(c) Local / Adaptive Thresholding (Niblack's Method)

Implementation and Optimization

To overcome the limitations of global thresholding, Niblack's adaptive thresholding calculates a unique threshold $T(x, y)$ for every individual pixel based on the statistical distribution of its local $W \times W$ spatial neighborhood. The threshold is defined mathematically as:

$$T(x, y) = \mu(x, y) + k \cdot \sigma(x, y)$$

where μ is the local mean, σ is the local standard deviation, and k is a user-defined weighting constant.

To prevent the algorithm from exhibiting $O(W^2)$ time complexity per pixel, the C++ engine utilizes Integral Images (Summed-Area Tables). By pre-computing a 2D integral sum matrix and an integral squared-sum matrix in a single pass, the local mean and variance for any arbitrarily sized window can be retrieved in $O(1)$ constant time, dramatically accelerating execution.

Parameter Tuning and Observations

Adaptive thresholding successfully resolved the complex lighting gradients that defeated the global methods, though precise parameter tuning (W and k) was essential to optimize each distinct scenario and eliminate mathematical artifacts.

- **Receipt** ($W = 161$, $k = -0.8$): Previous iterations with small windows introduced a classic Niblack artifact: "ghosting" or concentric rings. Because the receipt contains vast expanses of uniform white paper, the local standard deviation (σ) approaches zero, causing microscopic digital noise to cross the threshold and manifest as heavy black speckling. By drastically increasing the window to 161 and lowering k to -0.8 , the threshold is safely pushed below the noise floor, cleanly extracting the text despite the edge shadow.
- **Lilavati Manuscript** ($W = 101$, $k = -0.7$): Increasing the window size to 101 prevents the local standard deviation from collapsing inside the large dark water stains. Niblack's method successfully adapts to treat these stains as local background while preserving the foreground ink. The historical text was completely recovered without the massive black void artifacts seen in Otsu's method.
- **Blackboard** ($W = 81$, $k = 0.5$, **inverted**): To properly handle the inverted logic (where pixels $> T$ are foreground), a positive $k = 0.5$ was applied to push the threshold safely above the background noise floor. This larger 81×81 window spans the thick, cursive handwriting without hollowing out the strokes, successfully capturing both the bright text on the right and the faint text on the left while suppressing the chalk dust.
- **QR Code** ($W = 91$, $k = -0.2$): A massive 91×91 window is necessary to prevent the large black alignment squares from becoming hollowed out. By adapting locally, Niblack's method completely neutralizes the severe diagonal glare, perfectly segmenting the data blocks in both the washed-out and shadowed corners where global methods failed.

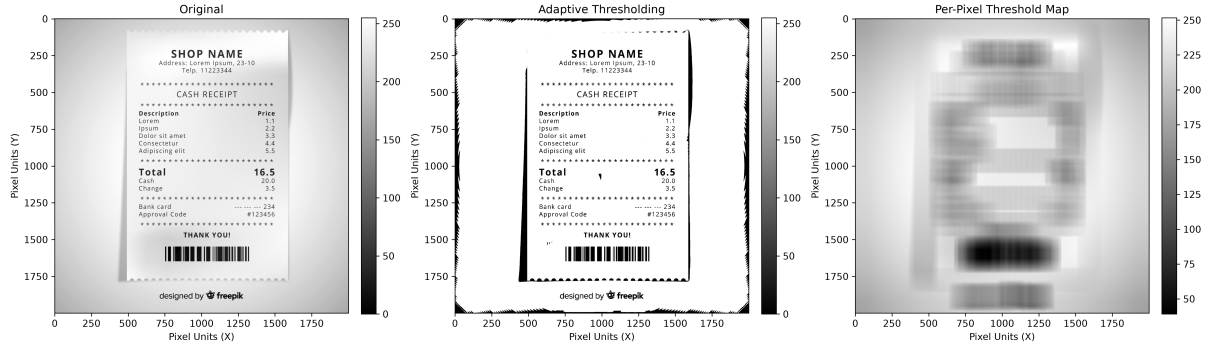


Figure 1: Adaptive Thresholding of `receipt.png` ($W = 161$, $k = -0.8$). The increased window size and strict negative offset successfully eliminate the background ghosting artifacts.

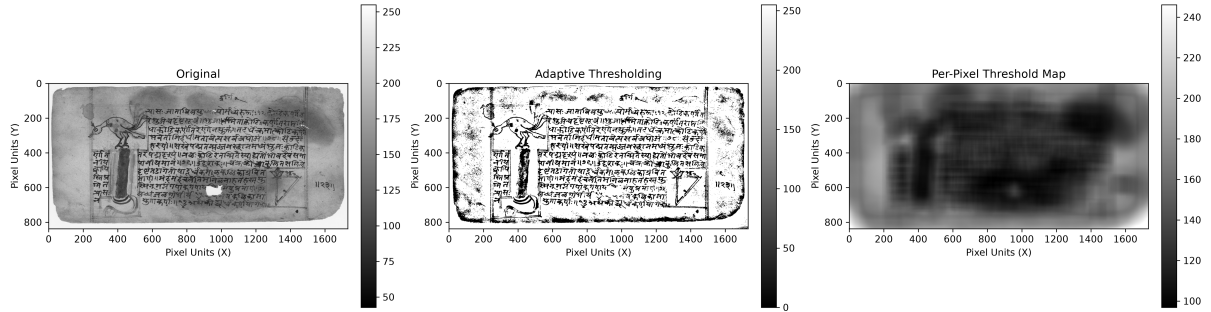


Figure 2: Adaptive Thresholding of `lilavati.png` ($W = 101$, $k = -0.7$). The local window adapts over the water stains to preserve the structural integrity of the text.

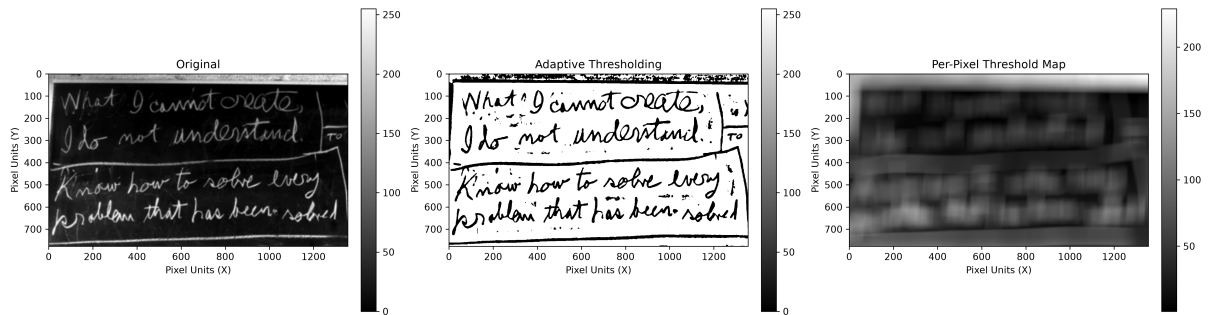


Figure 3: Adaptive Thresholding of `blackboard.png` ($W = 81$, $k = 0.5$, inverted). The positive offset prevents background chalk dust from being misclassified as text.

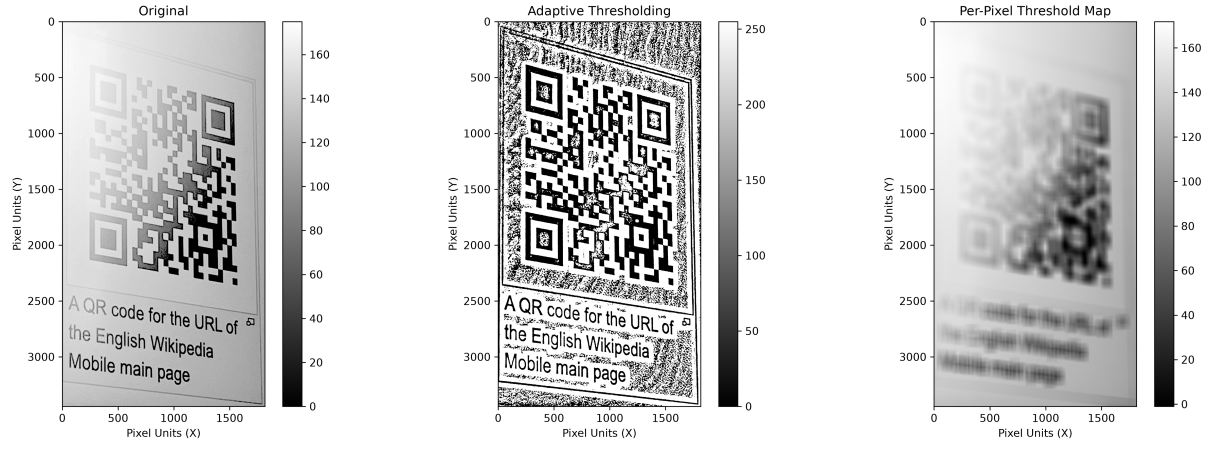


Figure 4: Adaptive Thresholding of `qr.png` ($W = 91$, $k = -0.2$). The per-pixel threshold map directly mimics the diagonal lighting glare to isolate the foreground modules.